

P24DS156 – Massive Data Analytics Laboratory

Windows Environment Practical Examination Manual

Course Code : P24DS156

Course Title : Massive Data Analytics Laboratory

SOFTWARE REQUIREMENTS

1. Java JDK
2. Hadoop
3. Apache Spark
4. Python
5. MySQL
6. Sqoop
7. WinRAR / 7-Zip
8. VS Code or Eclipse
9. Windows 10 / Windows 11

BASIC COMMANDS

Start Hadoop Services

```
start-all.cmd  
jps
```

Open Spark

```
pyspark
```

Create HDFS Folder

```
hdfs dfs -mkdir /input
```

Upload File to HDFS

```
hdfs dfs -put input.txt /input
```

View HDFS File

```
hdfs dfs -cat /input/input.txt
```

EXERCISE – 1

Aim

Implement and execute a Word Count program for a text file using Hadoop Java JAR and analyze the generated output.

CO Mapping

CO2 – Develop parallel processing applications using MapReduce

.

STEP-BY-STEP PROCEDURE

Step 1: Create Input File

```
notepad input.txt
```

Input:

```
hadoop spark hadoop  
spark bigdata
```

.

Step 2: Upload Input File into HDFS

```
hdfs dfs -mkdir /input  
hdfs dfs -put input.txt /input
```

.

Step 3: Compile Java Program

```
javac WordCount.java
```

.

Step 4: Create JAR File

```
jar -cvf wordcount.jar *.class
```

.

Step 5: Run Hadoop Program

```
hadoop jar wordcount.jar WordCount /input /output
```

.

Step 6: Display Output

```
hdfs dfs -cat /output/part-r-00000
```

.

EXPECTED OUTPUT

```
bigdata 1  
hadoop 2  
spark 2
```

.

HOW TO VERIFY CORRECT ANSWER

1. Check Hadoop services:

```
jps
```

2. Verify output folder:

```
hdfs dfs -ls /output
```

3. Display result:

```
hdfs dfs -cat /output/part-r-00000
```

.

RESULT

Thus the Hadoop Word Count program was executed successfully.

```
=====
```

EXERCISE – 2

Aim

Create and display an RDD with sample data using Apache Spark and examine its elements.

CO Mapping

CO3 – Analyse and visualize large datasets using Spark

STEP-BY-STEP PROCEDURE

Step 1: Open PySpark

```
pyspark
```

Step 2: Create RDD

```
data = [10,20,30,40,50]  
rdd = sc.parallelize(data)
```

Step 3: Display Elements

```
rdd.collect()
```

EXPECTED OUTPUT

```
[10, 20, 30, 40, 50]
```

HOW TO VERIFY CORRECT ANSWER

1. RDD should be created without error.
2. collect() function should display all elements.
3. Output must match inserted values.

RESULT

Thus the RDD was created and displayed successfully.

```
=====
===
```

EXERCISE – 3

Aim

Demonstrate the process of importing MySQL table data into HDFS using Sqoop and verify the transferred data.

CO Mapping

CO1 – Perform distributed data storage and processing using Hadoop

STEP-BY-STEP PROCEDURE

Step 1: Create Database and Table

```
CREATE DATABASE studentdb;  
USE studentdb;
```

```
CREATE TABLE student(  
id INT,  
name VARCHAR(20)  
);
```

Step 2: Insert Data

```
INSERT INTO student VALUES(1, 'Arun');  
INSERT INTO student VALUES(2, 'Kumar');
```

Step 3: Execute Sqoop Import

```
sqoop import --connect jdbc:mysql://localhost/studentdb --username root --  
password root --table student --target-dir /sqoop/student
```

Step 4: Verify Imported Data

```
hdfs dfs -cat /sqoop/student/part-m-00000
```

EXPECTED OUTPUT

1, Arun
2, Kumar

HOW TO VERIFY CORRECT ANSWER

1. Check whether HDFS folder exists.
2. Verify imported records.
3. Compare MySQL records with HDFS records.

RESULT

Thus the MySQL data was imported into HDFS successfully.

=====

EXERCISE – 4

Aim

Process streaming data using Apache Spark Streaming for real-time analytics.

CO Mapping

CO4 – Process streaming data for real-time analytics

.

STEP-BY-STEP PROCEDURE

Step 1: Start Netcat

```
nc -lk 9999
```

.

Step 2: Open PySpark

```
pyspark
```

.

Step 3: Execute Program

```
from pyspark.streaming import StreamingContext
```

```
ssc = StreamingContext(sc,5)
lines = ssc.socketTextStream("localhost",9999)
words = lines.flatMap(lambda line: line.split(" "))
words.pprint()
```

```
ssc.start()
ssc.awaitTermination()
```

.

Step 4: Enter Data in Netcat Window

```
hello spark
stream analytics
```

.

EXPECTED OUTPUT


```
hello
spark
stream
analytics
.
```

HOW TO VERIFY CORRECT ANSWER

1. Spark should continuously receive data.
2. Words typed in Netcat should appear in Spark output.
3. Streaming process must execute without error.

RESULT

Thus Spark Streaming was executed successfully.

```
=====
===
```

EXERCISE – 5

Aim

Develop and run a Word Count application for a text file using Hadoop Streaming with Python and interpret the result.

CO Mapping

CO2 – Develop parallel processing applications using MapReduce

MAPPER PROGRAM

```
import sys

for line in sys.stdin:
    words = line.split()
    for word in words:
        print(f"{word}\t1")
```

REDUCER PROGRAM

```
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.split('\t')
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(current_word, current_count)
            current_word = word
            current_count = count

if current_word:
    print(current_word, current_count)
```

EXECUTION COMMAND

```
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar -mapper  
mapper.py -reducer reducer.py -input /input/input.txt -output /stream_output
```

.

VIEW OUTPUT

```
hdfs dfs -cat /stream_output/part-00000
```

.

EXPECTED OUTPUT

```
hadoop 2  
spark 2  
bigdata 1
```

.

RESULT

Thus Hadoop Streaming Word Count using Python was executed successfully.

```
=====
```

EXERCISE – 6

Aim

Evaluate data trends and patterns using Spark visualization tools.

CO Mapping

CO5 – Evaluate data trends and patterns with advanced visualization tools

.

PROGRAM

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DataAnalysis").getOrCreate()

data = [("Jan",100),("Feb",200),("Mar",300)]

df = spark.createDataFrame(data,["Month","Sales"])

df.show()
```

.

EXPECTED OUTPUT

```
+-----+-----+
|Month|Sales|
+-----+-----+
| Jan | 100 |
| Feb | 200 |
| Mar | 300 |
+-----+-----+
```

.

RESULT

Thus the data trends were analyzed successfully using Spark.

```
=====
===
```

EXERCISE – 7

Aim

Execute and validate the export of data from HDFS to a MySQL table using Sqoop.

CO Mapping

CO1 – Perform distributed data storage and processing using Hadoop

STEP-BY-STEP PROCEDURE

Step 1: Create MySQL Table

```
CREATE TABLE employee(  
id INT,  
name VARCHAR(20)  
);
```

Step 2: Export Data from HDFS

```
sqoop export --connect jdbc:mysql://localhost/studentdb --username root --  
password root --table employee --export-dir /sqoop/student
```

Step 3: Verify Data in MySQL

```
SELECT * FROM employee;
```

EXPECTED OUTPUT

```
1 Arun  
2 Kumar
```

RESULT

Thus the HDFS data was exported successfully into MySQL.

```
=====
```

EXERCISE – 8

Aim

Apply massive data analytics techniques using Hadoop or Spark to solve a real-world problem aligned with SDGs.

CO Mapping

CO6 – Apply massive data analytics techniques to solve real-world problems aligned with SDGs

.

EXAMPLE PROBLEM

Analyze student attendance data using Spark.

.

PROGRAM

```
attendance = [("Arun",85),("Kumar",92),("Ravi",70)]
```

```
rdd = sc.parallelize(attendance)
```

```
result = rdd.filter(lambda x:x[1] > 80)
```

```
print(result.collect())
```

.

EXPECTED OUTPUT

```
[('Arun', 85), ('Kumar', 92)]
```

.

RESULT

Thus the real-world analytics problem was solved successfully using Spark.

```
=====
===
```

EXERCISE – 9

Aim

Construct and demonstrate a Word Count program using Python and Hadoop Streaming to process text data efficiently.

CO Mapping

CO2 – Develop parallel processing applications using MapReduce

.

PROCEDURE

1. Create input.txt
2. Create mapper.py
3. Create reducer.py
4. Upload input file into HDFS
5. Execute Hadoop Streaming command
6. Display output

.

EXPECTED OUTPUT

hadoop 2
spark 2

.

RESULT

Thus the Word Count application using Hadoop Streaming was executed successfully.

=====
===

EXERCISE – 10

Aim

Perform and evaluate filter operation and word count using Apache Spark for a given dataset.

CO Mapping

CO3 – Analyse and visualize large datasets using Spark

.

PROGRAM

```
words = ["spark", "hadoop", "bigdata", "spark"]  
  
rdd = sc.parallelize(words)  
  
filtered = rdd.filter(lambda x:x == "spark")  
  
print(filtered.collect())
```

.

EXPECTED OUTPUT

```
['spark', 'spark']
```

.

RESULT

Thus Spark filter operation was performed successfully.

```
=====
```


EXERCISE – 11

Aim

Read, transform, and process a CSV file using RDD map operation in Apache Spark.

CO Mapping

CO3 – Analyse and visualize large datasets using Spark

.

CSV FILE

```
1,Arun
2,Kumar
3,Ravi
```

.

PROGRAM

```
rdd = sc.textFile("student.csv")

result = rdd.map(lambda x:x.split(","))

print(result.collect())
```

.

EXPECTED OUTPUT

```
[['1', 'Arun'], ['2', 'Kumar'], ['3', 'Ravi']]
```

.

RESULT

Thus CSV file processing using Spark RDD map operation was executed successfully.

```
=====
===
```

EXERCISE – 12

Aim

Implement a streaming data processing application using Apache Spark Streaming for real-time data analysis.

CO Mapping

CO4 – Process streaming data for real-time analytics

.

PROGRAM

```
from pyspark.streaming import StreamingContext

ssc = StreamingContext(sc,10)

lines = ssc.socketTextStream("localhost",9999)

lines.pprint()

ssc.start()
ssc.awaitTermination()
.
```

EXPECTED OUTPUT

```
real time analytics
spark streaming
.
```

RESULT

Thus Spark Streaming real-time processing was implemented successfully.

```
=====
===
```

VIVA QUESTIONS

1. What is Hadoop?
2. What is MapReduce?
3. Define RDD.
4. What is Spark?
5. What is Sqoop?
6. Difference between Hadoop and Spark.
7. What is HDFS?
8. What is Spark Streaming?
9. Define Mapper and Reducer.
10. What is Big Data?

=====

===

COMMON ERRORS AND SOLUTIONS

Error	Solution
Hadoop services not running	Execute start-all.cmd
Output directory already exists	Delete using <code>hdfs dfs -rm -r /output</code>
Python file not found	Check mapper.py and reducer.py path
Spark not opening	Verify SPARK_HOME and JAVA_HOME
Sqoop connection failed	Verify MySQL username and password

=====

===